

LunarSight Multi-Agent Architecture – Detailed Educational Rewrite

Overview

This report is a pedagogical expansion of the original "LunarSight" document, keeping the same seven-chapter structure but explaining each concept in more detail and in more student-friendly language.^[^1] It focuses on how the five agents and the LangGraph orchestration work together to detect lunar subsurface ice in the South Pole region and plan safe rover traverses.

Each chapter has two layers:

- A detailed technical explanation (for deep understanding and implementation).
- A short "How to understand it" section written in simpler terms for quick intuition.

Where appropriate, suggested diagrams and external links to simulations or learning resources are included so the architecture can be studied and prototyped.

Chapter 1 – Agent 1: Data Ingestion and Topographic Synthesis

Technical explanation

Agent 1 is the "data engineer" of the LunarSight system. Its job is to fetch raw orbital radar data and elevation models, align them precisely in a common coordinate system, and derive slope and aspect maps needed by downstream physics and path-planning agents.^[^1]

Data sources and APIs

- ISRO Science Data Archive (ISDA) via the PRADAN portal provides Chandrayaan-2 DFSAR data in PDS4 format (L-band and S-band complex radar swaths).
- NASA Planetary Data System (PDS) LOLA Data Node provides global lunar elevation data with ~10 cm vertical precision and ~1 m absolute accuracy at high latitudes.^[^1]

Agent 1 authenticates to both APIs, queries data using a bounding box (e.g., Faustini Crater coordinates), and downloads the relevant `.img` and `.tif` files to local NVMe storage for high-throughput processing.^[^1]

Geospatial processing stack

- GDAL + Rasterio: for reading, warping, and reprojecting multi-gigabyte raster arrays.
- NumPy/SciPy: for vectorized numeric operations on elevation and radar grids.
- PDS4 client libraries: for parsing XML labels that describe radar products (dimensions, scaling, polarization channels, etc.).^[^1]

All rasters are reprojected into a unified lunar polar stereographic CRS, referenced to a 1737.4 km lunar sphere, so each radar pixel corresponds to the same physical location as the LOLA DEM pixel.[^1]

Horn's slope and aspect algorithm

Raw elevation is not directly useful for rover safety; Agent 1 must convert DEMs into quantitative slope and aspect fields.[^1]

Horn's method uses a 3×3 moving window over the DEM:

- Computes east–west gradient

$$\frac{dz}{dx}$$

using weighted differences between the top and bottom rows.

- Computes north–south gradient

$$\frac{dz}{dy}$$

using weighted differences between the left and right columns.

- Derives slope angle

$$\theta$$

from the magnitude of these gradients:

$$\theta = \arctan \left(\sqrt{(dz/dx)^2 + (dz/dy)^2} \right)$$

[^1]

- Computes aspect (direction the slope faces) from

$$\arctan(dz/dy, dz/dx)$$

[^1]

Horn's 3×3 weighted window is chosen because it smooths high-frequency noise and step artefacts inherent in laser altimetry, which is crucial in heavily cratered polar terrain.[^1]

Internal flow of Agent 1

1. Read global state for target crater bounding box (lat/long range).
2. Authenticate to ISDA PRADAN and NASA PDS; query DFSAR and LOLA datasets intersecting the box.[^1]
3. Download products to NVMe storage.
4. Reproject radar and DEM grids into a unified polar stereographic CRS.
5. Apply vectorized Horn's algorithm to produce slope and aspect rasters.
6. Write a multi-channel tensor to shared memory: [S-band complex, L-band complex, DEM, slope].
7. Update global state with file paths and `agent1_status = success` so the orchestration can trigger Agent 2.[^1]

Suggested diagrams

- Block diagram showing data flow from PRADAN + PDS → GDAL/Rasterio → DEM + slope tensor.
- 3×3 Horn window diagram illustrating weights and gradient directions.
- Coordinate reprojection diagram from original instrument geometry into polar stereographic CRS.

Suggested links / simulations

- GDAL Raster reprojection examples (terrain algorithms): <https://www.aazuspan.dev/blog/terrain-algorithms-from-scratch>
- Modeling slope in GIS (Horn's method): <https://scholarworks.uark.edu/context/jaas/article/1578/viewcontent/v58a16.pdf>

How to understand it (simple)

Think of Agent 1 as a very careful map maker:

- It downloads radar and height maps of the Moon.
- It lines them up perfectly so each pixel is in the right place.
- It converts height into "how steep" and "which way the slope faces" for every pixel.

Everything else in LunarSight trusts Agent 1's map. If this map is wrong, no other agent can do physics or path-planning safely.[^1]

Chapter 2 – Agent 2: Complex-Valued SAR Despeckling

Technical explanation

Agent 2 is the radar image "cleaner." Synthetic Aperture Radar (SAR) images suffer from speckle –multiplicative noise caused by coherent interference from many small scatterers within each pixel.[^1]

The standard multiplicative model is:

- $$Y = F \cdot X$$

, where
$$Y$$

is noisy intensity,

$$X$$

is true radar cross-section,

$$F$$

is a random speckle variable.[^1]

Classical filters (Lee, Frost, Gamma-MAP, etc.) reduce speckle by local statistics in a window, but they blur edges and often destroy polarimetric phase relationships needed to compute physically meaningful parameters such as Stokes vectors.[^1]

To avoid this, Agent 2 uses a Complex-Valued Convolutional Neural Network (CV-CNN) denoising autoencoder:

Complex-valued neural architecture

- Works on **complex tensors** (real + imaginary parts) instead of just amplitude.
 - Uses complex convolutions, complex activation functions (e.g., Complex-ReLU), and complex batch normalization.
 - Runs on PyTorch with `torch.complex64` and libraries like ComplexPyTorch, cvnn, accelerated by CUDA GPUs (H100/A100).
- [^1]

The input to Agent 2 is the complex radar data re-expressed as covariance matrices:

- Dual-pol: 2×2 covariance C_2
with diagonal intensity terms and off-diagonal complex cross-covariances.
- Quad-pol: 3×3 covariance C_3
for full polarimetry.

[^1]

The CV-CNN learns to map noisy covariance tensors to clean ones by minimizing a complex-aware loss function that respects both amplitude and phase, preserving the physics encoded in off-diagonal complex conjugate terms.[^1]

Internal flow of Agent 2

1. Read state to locate co-registered L-band and S-band radar arrays produced by Agent 1.
2. Compute C_2
 $/$ C_3
covariance matrices for each band.
3. Convert them into `torch.complex64` tensors.[^1]
4. Initialize CV-CNN with pre-trained weights on a GPU (e.g., H100/A100).
5. Run forward passes to remove speckle in the complex domain.
6. Verify polarimetric phase relationships are intact.
7. Serialize clean covariance tensor to disk and update global state for Agent 3.[^1]

Suggested diagrams

- Flow chart: noisy radar → covariance matrix → CV-CNN → clean covariance.
- Complex plane illustration showing why phase must be preserved.
- Covariance matrix representation highlighting diagonal vs off-diagonal terms.

Suggested links / simulations

- Complex vs real-valued CNN for PolSAR classification: <https://elib.dlr.de/193336/1/Complex-ValuedVs.Real-ValuedConvolutionalNeuralNetworkforPolsarDataClassification.pdf>
- ComplexPyTorch toolbox: <https://github.com/wavefrontshaping/complexPyTorch>
- SAR despeckling autoencoder example: <https://arxiv.org/html/2408.15678v2>

How to understand it (simple)

Think of Agent 2 as a noise removal filter that **does not ruin the physics**:

- Traditional filters blur the image and throw away important phase information.
- Agent 2 uses a complex-valued deep network that "listens" to both magnitude and phase of the radar wave.
- It outputs a cleaner radar representation where the true signal is preserved and speckle is minimized.

This clean complex data is necessary before doing polarimetry and feature extraction.[¹]

Chapter 3 – Agent 3: Polarimetric Feature Extraction

Technical explanation

Agent 3 is a physics engine that converts despeckled radar covariance matrices into physically interpretable features such as Stokes parameters, Circular Polarization Ratio (CPR), and m - χ decomposition components.[¹]

It uses NumPy/SciPy to perform deterministic, pixel-wise calculations over the entire radar field, relying on well-established planetary radar polarimetry theory.[¹]

Stokes parameters from covariance

Assuming a circular transmission basis, Agent 3 computes:

- S_1
: total power.
- S_2
,
 S_3
,
 S_4
: parameters describing the polarization ellipse.

[¹]

From these, it computes CPR:

- $$CPR = \frac{S_1 - S_4}{S_1 + S_4}$$

High CPR (> 1) is historically associated with volume scattering in water ice, but can also come from very rough surfaces, leading to ambiguity.^[1]

m- χ decomposition

To resolve rock vs ice ambiguity, Agent 3 uses Raney's m- χ decomposition:^[1]

- Degree of polarization

$$m = \frac{\sqrt{S_2^2 + S_3^2 + S_4^2}}{S_1}$$

- Ellipticity parameter

$$\chi$$

via

$$\sin(2\chi) = -\frac{S_4}{mS_1}$$

Using

$$m$$

and

$$\chi$$

, the total backscatter is decomposed into three physical scattering mechanisms, mapped to RGB:

- Red: even/double-bounce scattering (rocks, crater walls).
- Green: volumetric scattering (water ice in regolith).
- Blue: odd/surface scattering (flat regolith).

^[1]

Agent 3 also hardcodes empirical thresholds from Sinha et al. (2026):

- Flags pixels with CPR > 1 and DOP < 0.13 as strong candidates for doubly shadowed crater ice.

^[1]

Polarimetric feature tensor

All computed layers (CPR, m, RGB scattering components, thresholds) are stacked into a "Polarimetric Feature Tensor" that encodes rich physical information per pixel, ready for machine learning segmentation.

Internal flow of Agent 3

1. Read state to find clean L/S-band covariance tensors from Agent 2.
2. Compute Stokes parameters for each pixel.
3. Compute CPR and m.
4. Compute m- χ components (odd, even, volume) and map to RGB feature channels.

5. Apply Sinha et al. thresholds to flag candidate ice pixels.
6. Write Polarimetric Feature Tensor to disk and update global state.[^1]

Suggested diagrams

- Poincaré sphere visualization showing m and χ and how they map to scattering types.
- RGB mapping diagram: red (rocks), green (ice), blue (regolith).
- CPR vs DOP threshold plot showing ice vs rock clusters.

Suggested links / simulations

- Raney's m - χ decomposition paper: <https://www.researchgate.net/publication/258663012>
- ESA m - χ decomposition note: <http://earth.esa.int/eogateway/documents/20142/0/Raney-Methods.pdf>

How to understand it (simple)

Agent 3 turns the cleaned radar signal into **physical labels**:

- It figures out how much of the signal comes from flat ground, rough rocks, or ice hidden in the soil.
- It uses math to color-code every pixel into "rock-like", "ice-like", or "regolith-like".
- It also marks pixels that match strong ice conditions (CPR > 1 and low polarization).

The output is like a feature-rich, physics-based image that AI can understand.[^1]

Chapter 4 – Agent 4: Weakly-Supervised Semantic Segmentation

Technical explanation

Agent 4 is the main AI "vision" agent. It consumes the Polarimetric Feature Tensor from Agent 3 plus topographic data from Agent 1, and outputs a dense binary mask: Ice vs Rock for every pixel.[^1]

Because there is no dense ground truth drilling data in permanently shadowed regions, Agent 4 must rely on **weak supervision**.

Network architecture (SMP + U-Net)

- Uses the Segmentation Models PyTorch (SMP) library.
 - Deploys a U-Net architecture with skip connections, suited for high-resolution semantic segmentation.
 - Uses encoders like EfficientNet or ResNet pretrained on ImageNet for transfer learning.
- [^1]

Pseudo-label generation from physical priors

Agent 4 generates sparse pseudo-labels based on Agent 3's physics:

- Ice seeds (label 1) only where L-band CPR > 1, S-band CPR > 1, and DOP < 0.13.
 - Rock seeds (label 0) in sunlit equatorial pixels with strong odd-bounce (bare regolith).
 - Most pixels remain unlabeled (NaN), representing ambiguous territory.
- [^1]

During training, U-Net learns to expand from these sparse seeds to dense masks using contextual information in the multi-channel tensor (intensity, slope, m-χ features) via weakly supervised strategies such as CRFs and self-training.[^1]

Internal flow of Agent 4

1. Read Polarimetric Feature Tensor (Agent 3) and DEM/slope tensor (Agent 1) from state.
 2. Run heuristic mask generator to produce sparse ice/rock seeds using CPR and DOP thresholds.
 3. Initialize U-Net model via SMP with pre-trained encoder.
 4. Fine-tune on sparse labels, learning polarimetric and topographic textures.
 5. After convergence, run forward inference to produce a full Binary Ice Mask and confidence map.
 6. Write mask and confidence map to state as the geological target for Agent 5.
- [^1]

Suggested diagrams

- U-Net architecture diagram showing encoder/decoder and skip connections.
- Illustration of sparse seed labels being expanded into a dense mask.
- Layer stack diagram of the input tensor (CPR, m, RGB scattering, slope, etc.).

Suggested links / simulations

- SMP documentation: <https://segmentation-models-pytorch.readthedocs.io/en/latest>
- Weakly supervised segmentation in remote sensing: <https://www.mdpi.com/2072-4292/17/5/808>
- iSAGE human-in-the-loop framework: <https://arxiv.org/html/2606.10136v1>

How to understand it (simple)

Agent 4 is like an AI that learns "what ice looks like" using physics hints instead of full labels:

- Physics (from Agent 3) gives a few very reliable ice and rock pixels.
- The U-Net uses those as examples and learns patterns around them.
- It spreads these patterns across the whole crater, generating a detailed map of where ice probably is and where rock is.

The result is a high-resolution map that says "Here is subsurface ice" vs "Here is bare rock".[^1]

Chapter 5 – Agent 5: Terramechanic-Aware Spatiotemporal Pathfinding

Technical explanation

Agent 5 is the rover navigation brain. It takes the Binary Ice Mask and the slope/topography tensors, and computes safe, physically realistic traverses from a sunlit crater rim to target ice deposits and back.^[1]

Traditional 2D geometric pathfinding (Dijkstra, static A*) is not enough because lunar terrain is:

- Steep and covered with loose regolith.
- Subject to extreme temperature and moving shadows.

Agent 5 therefore uses a **Spatiotemporal Kinodynamic A*** algorithm with terramechanics.

Terramechanics: slip, sinkage, soil failure

The rover-wheel-regolith interaction is modeled using:

- Mohr-Coulomb failure criterion: maximum shear stress

$$\tau_{max} = c + \sigma \tan \phi$$

, where

$$c$$

is soil cohesion,

$$\sigma$$

is normal stress,

$$\phi$$

is internal friction angle (~30–35° for lunar soil).

- Janosi-Hanamoto shear equation:

$$\tau = c + (\sigma \tan \phi)(1 - e^{-j/K})$$

, where

$$j$$

is shear displacement and

$$K$$

shear modulus.

^[1]

Using slope from Agent 1 and rover parameters (mass, wheel design), Agent 5 predicts slip, sinkage, tractive effort, and drawbar pull for each candidate path segment. If required traction exceeds

$$\tau_{max}$$

, the terrain is flagged as an infinite-cost obstacle.

Spatiotemporal illumination and thermal constraints

Lunar polar regions have long synodic periods (~708 hours) and deep shadows. The rover must manage:

- Battery capacity.

- Thermal budget in ~25 K environments.

[^1]

Agent 5 extends search space to 4D:

$$(x, y, z, t)$$

:

- Tracks moving sun vectors using LOLA + illumination models.
- Uses a heuristic

$$h(n)$$

that accounts for path length, slope-induced speed reduction, and energy usage.

- Ensures the rover can reach ice and return to sunlit rim before depletion.

[^1]

Internal flow of Agent 5

1. Read Binary Ice Mask (Agent 4) and DEM/slope (Agent 1) from state.
2. Discretize terrain into a graph of nodes (pixels with associated elevation/slope).
3. For each neighbor, compute cost

$$g(n)$$

using terramechanics (Mohr-Coulomb + Janosi-Hanamoto) to estimate slip and traction.

4. Compute heuristic

$$h(n)$$

with distance and illumination/time constraints.

5. Run kinodynamic A* over 4D search space to find optimal safe path.
6. Output path as a set of geographic coordinates, timestamps, and expected slip ratios for the rover controller.

[^1]

Suggested diagrams

- Free-body diagram of rover wheel on slope showing forces, slip, and sinkage.
- 4D pathfinding diagram (space + time) with moving shadow boundaries.
- Cost-field heatmap overlay on DEM showing safe vs unsafe regions.

Suggested links / simulations

- Dynamic illumination-constrained path planning for lunar South Pole: <https://www.mdpi.com/2072-4292/18/23/310>
- Terramechanics overview: https://www.kiss.caltech.edu/workshops/xTerrapresentations/1_iagnema.pdf
- Drawbar pull vs slip analysis: <https://apps.dtic.mil/sti/pdfs/AD0275858.pdf>

How to understand it (simple)

Agent 5 is the agent that asks: "Can the rover actually drive there and survive?"

- It uses physics of soil and wheel traction to avoid slopes the rover cannot climb.
- It keeps track of battery and heat while planning the path.
- It searches in both space and time, choosing routes that are safe **and** reach ice while staying within power and thermal limits.

The output is a realistic driving plan rather than just a shortest geometric route.^[1]

Chapter 6 – Unified Multi-Agent Orchestration via LangGraph

Technical explanation

Agents 1–5 are powerful specialists but must be coordinated by an orchestration layer that handles state, sequencing, and error recovery. LunarSight uses **LangGraph** to model the system as a stateful multi-agent application with cyclical workflows.

StateGraph architecture

1. State schema (TypedDict)

- Central JSON-like object holds shared context: file paths (e.g., `raw_data_path`, `c2_matrix_path`), status flags (`agent1_status`, `path_failure`), rover constraints (max slope, battery), etc.
- Agents pass lightweight references (file paths, metadata) instead of huge tensors to avoid memory fragmentation.^[1]

2. Nodes

- Agents 1–5 implemented as Python functions that:
 - Read needed context from state.
 - Perform their specialized computation.
 - Write outputs and status codes back to state.
- Each node is a vertex in the LangGraph StateGraph.^[1]

3. Edges and conditional routing

- Edges encode workflow transitions.
- A central Supervisor node reads state, decides which agent to run next, and handles failures by routing backwards when needed.
- Crucially, LangGraph allows cycles, enabling iterative refinement and conflict resolution.^[1]

Orchestration flow

- Phase 1: Preparation

- Human operator provides mission parameters (crater coordinates, rover limits).
- Supervisor triggers Agent 1.
- Phase 2: Radar analysis
 - Agent 1 completes and updates state.
 - Supervisor triggers Agent 2 → Agent 3 sequentially.
- Phase 3: Segmentation and routing
 - Supervisor triggers Agent 4 to generate Binary Ice Mask.
 - Then triggers Agent 5 for pathfinding.

Cyclical conflict resolution example

If Agent 5 reports `path_failure` (no safe route due to terramechanics constraints):

- Supervisor reads the flag and routes back to Agent 4 via a conditional edge.
 - State is modified to relax confidence thresholds or select alternative shallower ice deposits.
 - Agent 4 recomputes map; Agent 5 tries routing again.
- [^1]

This closed-loop cycle continues until a feasible solution or global failure condition is reached, embodying resilient multi-agent behavior.

Suggested diagrams

- StateGraph diagram with nodes (Agents 1–5, Supervisor) and arrows showing forward and backward edges.
- State object schema diagram showing keys and how they change over time.
- Cycle example: Agent 5 failure → Supervisor → Agent 4 → Agent 5.

Suggested links / simulations

- LangGraph multi-agent workflows overview: <https://www.langchain.com/blog/langgraph-multi-agent-workflows>
- Supervisor-worker patterns with LangGraph: <https://callsphere.ai/blog/langgraph-multi-agent-system-supervisor-worker-patterns>
- Example contextual engineering pipeline using LangGraph: <https://github.com/FareedKhan-dev/contextual-engineering-guide>

How to understand it (simple)

LangGraph is the "conductor" of the multi-agent orchestra:

- The state object is the shared notebook that all agents read from and write to.
- The Supervisor decides which agent runs next and what to do when something goes wrong.
- Cycles let the system smartly retry with adjusted parameters instead of crashing.

So the whole LunarSight pipeline becomes an intelligent workflow, not just a one-shot chain of scripts.[^1]

Chapter 7 – Strategic Blueprint for Enhancement and Scale

Technical explanation

The original hackathon proposal had strong high-level ideas but several critical weaknesses: fabricated mathematics, vague metrics, and missing physical detail. This chapter outlines a blueprint to turn LunarSight into a mission-ready architecture.^[^1]

1. Remove fabricated math; enforce scientific rigor

Issues:

- Arbitrary equations (e.g., made-up polynomials, fake kinetic energy formulas) not grounded in any radar or orbital physics.^[^1]
- Non-standard units (e.g., sbps, mprs, mpes) instead of real engineering metrics.

Actions:

- Completely remove all hallucinated equations from documentation and code.
- Use only physically correct formulas from Chapters 3 and 5 (Raney's $m\text{-}\chi$, Mohr–Coulomb, Janosi–Hanamoto).
- Replace fictional metrics with standardized ones: MB/s for data throughput, TFLOPS for compute, ms for latency.^[^1]

2. Exploit decentralized GPU economics

Training CV-CNN (Agent 2) and U-Net (Agent 4) on full-resolution DFSAR swaths requires high VRAM (80 GB for H100/A100). Hyperscalers such as AWS are expensive; decentralized GPU marketplaces like [Vast.ai](#) or Spheron can significantly reduce costs.^[^1]

Example pricing (2026 indices):

- H100 PCIE 80 GB on [Vast.ai/Spheron](#) ~2.00–2.01 USD/hr.
 - H100 SXM5 80 GB on AWS ~6.88 USD/hr.
 - A100 80 GB on [Vast.ai](#) ~0.51 USD/hr.
 - RTX 4090 24 GB on [Vast.ai](#) ~0.35 USD/hr.
- ^[^1]

With ~500 USD, one can secure ~250 hours of H100 time via decentralized marketplaces, enough for serious experimentation.^[^1]

3. Hardcode latest empirical radar thresholds

The segmentation quality hinges entirely on the pseudo-labels used by Agent 4. Bad thresholds → bad masks.

Actions:

- Anchor pseudo-label generation to current literature, e.g., Sinha et al. (2026): volumetric ice seeds if and only if L-band CPR > 1 and DOP < 0.13.^[^1]
- Maintain a configurable but version-controlled threshold module so updates are traceable and scientifically justified.

4. Integrate continuous spatiotemporal dynamics for real missions

To control actual rovers like Pragyan or VIPER, Agent 5 must act as a dynamic digital twin:

- Node expansion must carry a dynamic battery variable that depletes faster under high slip (higher torque and power draw).
- LangGraph orchestrator must continually feed updated illumination models (708 h cycle) into Agent 5.

^[^1]

This turns the path planner into a live, adaptive controller aligned with real thermal and power telemetry.

5. Documentation, reproducibility, and open tooling

For long-term scale:

- Maintain open-source repositories for each agent with clear APIs and test datasets.
- Provide Jupyter/Colab notebooks that show step-by-step pipelines: e.g., PDS4 parsing → Horn's DEM → CV-CNN despeckling → m-χ features → SMP U-Net training → A* pathfinding.
- Version control physical models and thresholds with clear citations.

Suggested diagrams

- Cost comparison bar chart for different GPU providers.
- Threshold logic flow chart for pseudo-label generation.
- Digital-twin diagram showing feedback loop between rover telemetry, Agent 5, and LangGraph.

Suggested links / simulations

- Decentralized GPU marketplace Vast.ai: <https://vast.ai>
- Complex-Valued SAR classification toolbox: <https://github.com/XinyuanLiao/ComplexNN>
- Path planning algorithm reviews for planetary rovers: <https://www.mdpi.com/2072-4292/17/11/1924>

How to understand it (simple)

Chapter 7 is the "upgrade roadmap":

- Delete any fake math or units—only keep equations that match real physics.
- Use cheaper GPUs from decentralized platforms so you can actually train these models as a student team.

- Lock in good physics-based thresholds so the AI learns from correct examples.
- Make Agent 5 behave like a live twin of the rover, using real-time power and temperature.
- Write clean, reproducible code and notebooks so others can understand, review, and extend LunarSight.

If you follow this blueprint, LunarSight moves from "cool hackathon idea" to a credible architecture for future lunar missions.[^1]

References

1. [DOC-20260702-WA0000.pdf](#) - page-1